

SPRING TESTING EXERCISES

Exercise 1: Basic Unit Test for a Service Method

Explanation

Tests a simple service method with JUnit 5 and verifies that adding two numbers returns the expected result.

Source Code

CalculatorService.java

```
package com.cognizant.springtesting;

import org.springframework.stereotype.Service;

@Service
public class CalculatorService {
    public int add(int a, int b) {
        return a + b;
    }
}
```

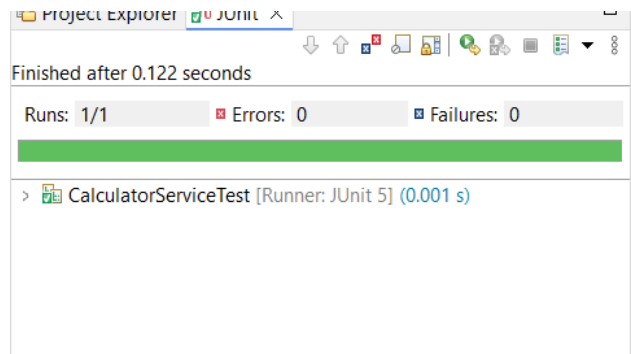
CalculatorServiceTest.java

```
package com.cognizant.springtesting;

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class CalculatorServiceTest {
    @Test
    public void testAdd() {
        CalculatorService service = new CalculatorService();
        assertEquals(15, service.add(10, 5));
    }
}
```

Output



Exercise 2: Mocking a Repository in a Service Test

Explanation

Uses Mockito to mock UserRepository. The repository response is stubbed so UserService can be tested without a real database.

Source Code

User.java

```
package com.cognizant.springtesting;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {
    @Id
    private Long id;
    private String name;

    public User() {}
    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

UserRepository.java

```
package com.cognizant.springtesting;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);
}
```

UserService.java

```
package com.cognizant.springtesting;

import java.util.NoSuchElementException;
import org.springframework.stereotype.Service;

@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
    public User saveUser(User user) {
        return userRepository.save(user);
    }
    public User getRequiredUser(Long id) {
```

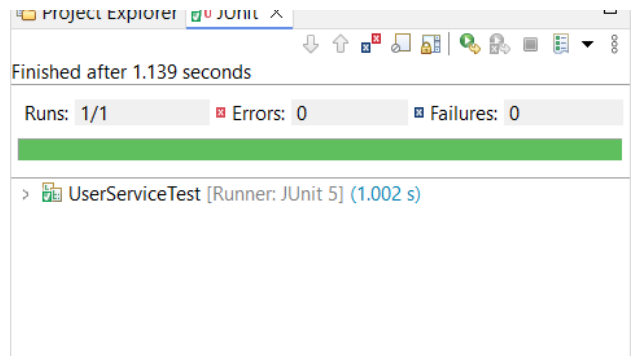
```
        return userRepository.findById(id)
            .orElseThrow(NoSuchElementException::new);
    }
}
```

UserServiceTest.java

```
package com.cognizant.springtesting;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import java.util.Optional;
import org.junit.jupiter.api.Test;

public class UserServiceTest {
    @Test
    public void testGetUserById() {
        UserRepository repo = mock(UserRepository.class);
        when(repo.findById(1L)).thenReturn(Optional.of(new User(1L, "Rekha")));
        User result = new UserService(repo).getUserById(1L);
        assertEquals("Rekha", result.getName());
        verify(repo).findById(1L);
    }
}
```

Output



Exercise 3: Testing a REST Controller with MockMvc

Explanation

Tests the REST controller with Spring MockMvc. UserService is mocked and the HTTP status and JSON response are verified.

Source Code

UserController.java

```
package com.cognizant.springtesting;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        return ResponseEntity.ok(userService.saveUser(user));
    }
}
```

UserControllerTest.java

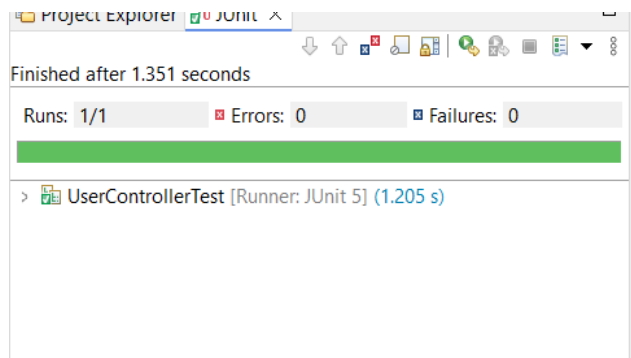
```
package com.cognizant.springtesting;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(UserController.class)
public class UserControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean UserService userService;

    @Test
    void testGetUser() throws Exception {
        when(userService.getUserById(1L)).thenReturn(new User(1L, "Rekha"));
        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Rekha"));
    }
}
```

Output



Exercise 4: Integration Test with Spring Boot

Explanation

Loads the Spring Boot application context and tests the complete controller-to-service-to-repository flow.

Source Code

SpringIntegrationTest.java

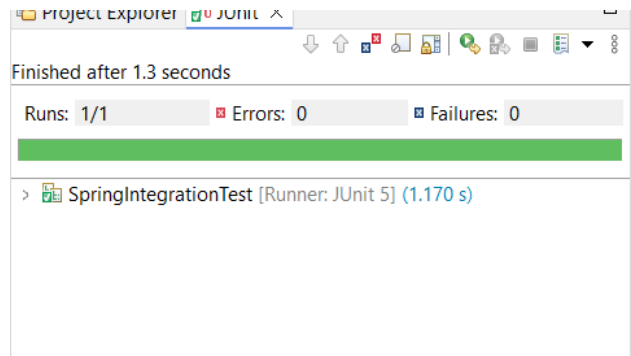
```
package com.cognizant.springtesting;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
public class SpringIntegrationTest {
    @Autowired MockMvc mockMvc;
    @Autowired UserRepository repository;

    @BeforeEach
    void setup() {
        repository.deleteAll();
        repository.save(new User(1L, "Rekha"));
    }

    @Test
    void testCompleteUserFlow() throws Exception {
        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Rekha"));
    }
}
```

Output



Exercise 5: Test Controller POST Endpoint

Explanation

Tests the POST endpoint that creates a user. JSON input is sent with MockMvc and the returned user is verified.

Source Code

UserController.java

```
package com.cognizant.springtesting;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        return ResponseEntity.ok(userService.saveUser(user));
    }
}
```

UserControllerPostTest.java

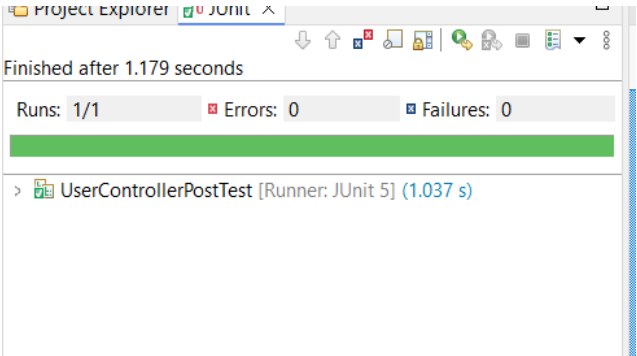
```
package com.cognizant.springtesting;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(UserController.class)
public class UserControllerPostTest {
    @Autowired MockMvc mockMvc;
    @MockBean UserService userService;

    @Test
    void testCreateUser() throws Exception {
        when(userService.saveUser(any(User.class))).thenReturn(new User(1L, "Rekha"));
        mockMvc.perform(post("/users").contentType(MediaType.APPLICATION_JSON)
            .content("{\"id\":1,\"name\":\"Rekha\"}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Rekha"));
    }
}
```

Output



Exercise 6: Test Service Exception Handling

Explanation

Verifies service exception handling when a requested user is missing. JUnit checks that `NoSuchElementException` is thrown.

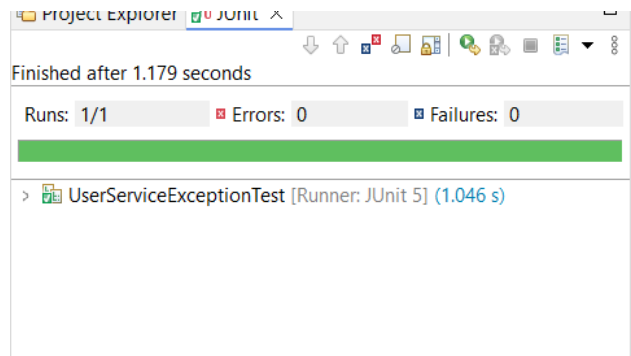
Source Code

UserServiceExceptionTest.java

```
package com.cognizant.springtesting;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;
import java.util.*;
import org.junit.jupiter.api.Test;

public class UserServiceExceptionTest {
    @Test
    void testMissingUser() {
        UserRepository repo = mock(UserRepository.class);
        when(repo.findById(99L)).thenReturn(Optional.empty());
        UserService service = new UserService(repo);
        assertThrows(NoSuchElementException.class,
            () -> service.getRequiredUser(99L));
        verify(repo).findById(99L);
    }
}
```

Output



Exercise 7: Test Custom Repository Query

Explanation

Tests the custom repository method `findByName()`. The mock repository returns matching users and the result is verified.

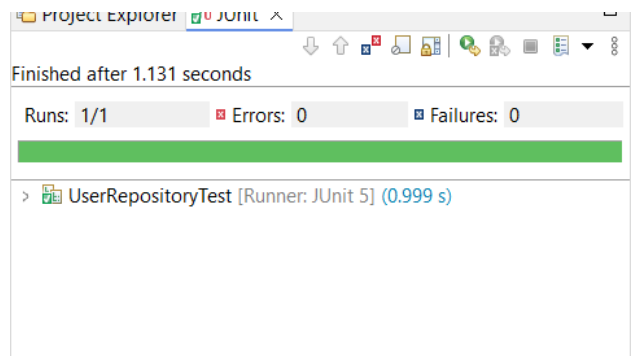
Source Code

UserRepositoryTest.java

```
package com.cognizant.springtesting;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import java.util.*;
import org.junit.jupiter.api.Test;

public class UserRepositoryTest {
    @Test
    void testFindByName() {
        UserRepository repo = mock(UserRepository.class);
        List<User> users = Arrays.asList(
            new User(1L, "Rekha"), new User(2L, "Rekha"));
        when(repo.findByName("Rekha")).thenReturn(users);
        List<User> result = repo.findByName("Rekha");
        assertEquals(2, result.size());
        verify(repo).findByName("Rekha");
    }
}
```

Output



Exercise 8: Test Controller Exception Handling

Explanation

Tests centralized exception handling with `@ControllerAdvice` and verifies the 404 response and error message.

Source Code

UserController.java

```
package com.cognizant.springtesting;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        return ResponseEntity.ok(userService.saveUser(user));
    }
}
```

GlobalExceptionHandler.java

```
package com.cognizant.springtesting;

import java.util.NoSuchElementException;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<String> handleNotFound(NoSuchElementException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not found");
    }
}
```

ControllerExceptionTest.java

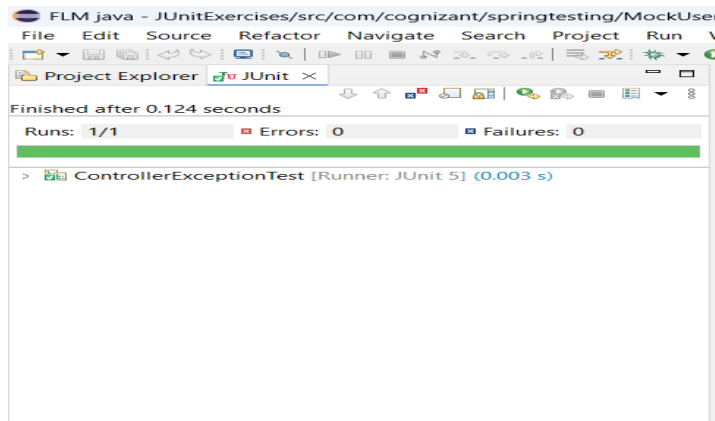
```
package com.cognizant.springtesting;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import java.util.NoSuchElementException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.context.annotation.Import;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(UserController.class)
@Import(GlobalExceptionHandler.class)
```

```
public class ControllerExceptionTest {  
    @Autowired MockMvc mockMvc;  
    @MockBean UserService userService;  
  
    @Test  
    void testUserNotFound() throws Exception {  
        when(userService.getUserById(99L)).thenReturn(new NoSuchElementException());  
        mockMvc.perform(get("/users/99"))  
            .andExpect(status().isNotFound())  
            .andExpect(content().string("User not found"));  
    }  
}
```

Output



Exercise 9: Parameterized Test with JUnit

Explanation

Uses `@ParameterizedTest` and `@CsvSource` to execute the same addition test with multiple sets of input values.

Source Code

CalculatorParameterizedTest.java

```
package com.cognizant.springtesting;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

public class CalculatorParameterizedTest {
    @ParameterizedTest
    @CsvSource({"1,2,3","5,5,10","10,20,30","-5,5,0"})
    void testAdd(int a, int b, int expected) {
        assertEquals(expected, new CalculatorService().add(a,b));
    }
}
```

Output

